DataWise — Usage Guide

How to run DataWise and use every major feature (tabular ML cleaning, documents, images, audio, video, archives). Written so anyone can follow the steps and re-run the same tests.

A shareable PDF copy of this guide is available as `USAGE_GUIDE.pdf` in the repo root (regenerate with `python scripts/export_usage_guide_pdf.py`).

1. Start the app

From the repo root (`DataWise/`):

```
# One-time (if not done): create venv + install
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt

# Every day — easiest
.\start.ps1
```

Or two terminals:

```
# Terminal 1 — API
.venv\Scripts\Activate.ps1
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 --workers 1

# Terminal 2 — UI
.venv\Scripts\Activate.ps1
streamlit run frontend/app.py --server.port 8501
```

Open:

Service	URL
UI	http://localhost:8501
API health	http://127.0.0.1:8000/health
API docs	http://127.0.0.1:8000/docs

Important: always use `--workers 1` for the API.

2. General workflow (all file types)

- Open the UI.
- Upload one (or more) files.
- Type a plain-English prompt.
- Click **Run Pipeline**.
- Review preview / summary / plan steps.
- Download the result.

Same idea via API:

```
curl -X POST "http://127.0.0.1:8000/api/pipeline/run" `
  -F "prompt=clean this for ML" `
  -F "file=@test_files/tabular/messy_customers.csv"
```

3. Tabular files (CSV / JSON / Excel) — ML cleaning

Test files

File	Path
Messy CSV	`test_files/tabular/messy_customers.csv`
Messy JSON	`test_files/tabular/messy_customers.json`
Messy Excel	`test_files/tabular/messy_customers.xlsx`

Also see older samples under `test_data/` and `tests/fixtures/sample.csv`.

What `clean_for_ml` does

When you ask to clean tabular data, DataWise:

- Strips whitespace
- Turns `N/A` / `null` / blanks / `-` / `?` into missing values
- Drops empty columns
- Removes duplicate rows
- Coerces numeric text → numbers
- Imputes remaining nulls (median / mode)
- Standardizes dates to `YYYY-MM-DD`
- Drops constant (no-signal) columns

Prompts to try

Prompt	Expected
`clean this for ML`	Full ML prep pipeline
`clean this CSV so I can use it in my ML project`	Same
`prepare this data for machine learning`	Same
`sort by Age ascending`	Sort only (all rows)
`show only Name and Age columns`	Column projection
`only display 2 columns`	Keep first 2 columns
`filter rows where Age > 30`	Row filter

Prompt	Expected
`group by Region and sum Purchase`	Groupby aggregate
`top 5 by Purchase`	Sort + top N
`remove the Phone column`	Drop column (if present)

How to test (manual)

- Upload `test_files/tabular/messy_customers.csv`.
- Prompt: `clean this for ML`.
- Confirm rows drop (dupes gone), empty/constant columns removed, download works.
- Repeat with `.json` and `.xlsx`.

How to test (API smoke)

```
.venv\Scripts\Activate.ps1
python -c "import httpx; from pathlib import Path; p=Path('test_files/tabular/messy_customers.csv'); r=httpx.post(
```

4. Documents (PDF / DOCX / TXT / MD)

Test files

Generate (or refresh) fixtures:

```
.venv\Scripts\python.exe scripts\generate_document_test_files.py
```

File	Path	Best for
PDF report	`test_files/documents/sample_report.pdf`	text, entities, pages, links, metadata
DOCX invoice	`test_files/documents/sample_invoice.docx`	tables→CSV, headings, key-values
TXT report	`test_files/documents/sample_report.txt`	entities, redact, summary
Markdown	`test_files/documents/sample_overview.md`	headings, links

Features and prompts

Feature	Example prompt	Notes
Extract text	`extract text from this document`	Default baseline
Clean text	`clean this document text`	Whitespace + header/footer noise

Feature	Example prompt	Notes
Word count	`count the words in this document`	
Metadata	`extract document metadata`	pages, author, size, etc.
Headings / outline	`extract headings` / `show the outline`	PDF/DOCX/MD
Tables → CSV	`extract all tables`	Writes `table_1.csv`, … in session folder
Split by page	`split this PDF by page`	Writes `page_1.txt`, …
Entities	`extract emails, phones, dates, and amounts`	Deterministic regex
Key-value / invoice fields	`extract invoice fields` / `extract key values`	`Label: value` lines
Find mentions	`find all mentions of "Revenue"`	Quotes recommended
Links	`extract all links and urls`	URLs + emails
Language	`detect the language of this document`	Lightweight heuristic
Images / figures	`extract images from this PDF`	PDF only
Redact PII	`redact emails and phone numbers`	Masks PII in text
Summary	`summarize this document`	Extractive (no LLM required)
Q&A	`what is the invoice number?`	Uses LLM if API key set; else returns extracted text fallback
OCR (scanned PDF)	`OCR this scanned PDF`	Needs **Tesseract** + `pytesseract`
Convert / export	`convert this document to markdown`	Download as txt/md/json/docx/pdf

How to test each document feature (checklist)

- Start backend + UI.
- Upload `test_files/documents/sample_invoice.docx`.
- Run: `extract all tables` → download / check session for CSV artifacts.
- Upload `sample_report.pdf`.
- Run in turn:
- `extract text from this document`
- `extract emails, phones, dates, and amounts`
- `extract invoice fields`

- `find all mentions of "Revenue"`
- `split this PDF by page`
- `extract document metadata`
- `summarize this document`
- Upload `sample_report.txt` → `redact emails and phone numbers`.
- Upload `sample_overview.md` → `extract headings`.

OCR note

OCR is optional. If Tesseract is not installed, the OCR step fails with a clear message. Text-layer PDFs work without OCR.

5. Images

Supported ops (see also `EXAMPLE_COMMANDS.md`):

Prompt example	Result
`resize this image to 1024x1024`	Resize
`crop image to 50, 50, 450, 350`	Crop
`rotate this image by 90 degrees`	Rotate
`convert image to PNG`	Format convert
`compress this image with quality 50`	Compress
`extract image metadata and dimensions`	Metadata

Fixtures: `tests/fixtures/images/` (generate via `scripts/generate_image_fixtures.py` if needed).

6. Audio

Prompt example	Result
`extract audio metadata`	Duration, sample rate, channels
`trim this audio to first 2 seconds`	Trim
`convert audio to mp3`	Convert
`compress this audio`	Compress

Requires **FFmpeg** on PATH (`ffmpeg -version`).

7. Video

Prompt example	Result
`extract video metadata`	Metadata
`trim this video to first 3 seconds`	Trim
`extract frame at 1 second`	Frame image
`extract audio from this video`	Audio track
`resize this video`	Resize (when planned)

Requires **FFmpeg**.

8. Archives (ZIP / TAR / GZ)

Prompt example	Result
`list files in this archive`	Member listing
`extract and process all files in archive`	Safe extract + process

Security: path traversal / zip-bomb guards are built in.

9. Automated tests (developers)

```
.venv\Scripts\Activate.ps1

# Full suite
.venv\Scripts\pytest.exe -q --tb=short

# Useful subsets
.venv\Scripts\pytest.exe backend/tests/test_tools_clean.py -q
.venv\Scripts\pytest.exe backend/tests/test_document_processor.py -q
.venv\Scripts\pytest.exe backend/tests/test_pipeline.py -q
.venv\Scripts\pytest.exe backend/tests/test_regression_emergency_fix.py -q
```

10. Feature map (quick reference)

Area	Main capability	Test folder
Tabular ML clean	`clean_for_ml`	`test_files/tabular/`
Documents	extract / tables / entities / redact / summary / …	`test_files/documents/`
Images	resize, crop, rotate, convert, compress	`tests/fixtures/images/`
Audio / video	metadata, trim, convert (FFmpeg)	`tests/fixtures/audio/`, `tests/fixtures/video/`
Archives	list / safe extract	`tests/fixtures/archives/`

11. Related docs

Doc	Purpose
`PROJECT_SETUP.md`	Install from scratch
`PROJECT_OVERVIEW.md`	Architecture
`EXAMPLE_COMMANDS.md`	More prompt examples
`capabilities.json`	Enabled processors / limits
`test_files/README.md`	Index of tabular + document fixtures
`test_files/documents/README.md`	Document fixtures

12. Troubleshooting

Problem	Fix
UI cannot reach API	Start uvicorn on port 8000; check `DATAWISE_API_URL` in `.env`
Weird session errors	Restart API with `--workers 1`
Audio/video fails	Install FFmpeg; reopen terminal
Document OCR fails	Install Tesseract + `pip install pytesseract`
Clean prompt does nothing unexpected	Use clearer prompts from this guide
Huge upload rejected	Confirm `.streamlit/config.toml` has `maxUploadSize = 10000`